

实验6.3 TCP拥塞控制-3

TCP拥塞避免算法与超时重传机制分析

慢启动、拥塞避免、拥塞处理和超时重传

- 慢启动算法分析
- 拥塞处理（超时）分析
- 拥塞避免算法分析
- 超时重传机制分析

拥塞避免算法实验

- 拥塞避免算法
  - 分析send3***文件的超时重传后的报文传输
  - 如何判断拥塞避免开始？
  - 拥塞窗口的变化与慢启动阶段有何不同？
  - 与snd_ssthresh的关系

拥塞避免算法

pkt_seqno	pkt_type	RorS_seqno	snd_ssthresh	snd	_wnd	snd_wnd_left	snd_wnd_pointer	snd_wnd_left+cwnd
4075	snd_data	2488	23	20	35	155458344	155485856	155487544
4076	snd_data	2489	23	20	35	155458344	155487304	155487544
4077	rcv_ack	1588	23	21	87	155459792	155487304	155490452
4078	snd_data	2490	23	21	87	155459792	155488752	155490452
4079	snd_data	2491	23	21	87			
4080	rcv_ack	1589	23	22	35			
4081	snd_data	2492	23	22	35			
4082	snd_data	2493	23	22	35			
4083	rcv_ack	1590	23	23	35			
4084	snd_data	2494	23	23	35			
4085	snd_data	2495	23	23	35	155464136	155495992	155497716
4086	snd_data	2496	23	23	35	155464136	155497440	155497716
4087	rcv_ack	1591	23	24	64087	155465584	155497440	155500624
4088	snd_data	2497	23	24	64087	155465584	155498888	155500624
4089	snd_data	2498	23	24	64087	155465584	155500336	155500624
4090	rcv_ack	1592	23	24	65535	155467032	155500336	155502072
4091	snd_data	2499	23	24	35	155467032	155501784	155502072
4092	rcv_ack	1593	23	24	35	155469928	155501784	155504968
4093	snd_data	2500	23	24	35	155469928	155503232	155504968
4094	snd_data	2501	23	24	35	155469928	155504680	155504968
4095	rcv_ack	1594	23	24	87	155471376	155504680	155506416
4096	snd_data	2502	23	24	87			
4097	rcv_ack	1595	23	24	35			
4098	snd_data	2503	23	24	35			
4099	rcv_ack	1596	23	24	35			
4100	snd_data	2504	23	24	35			
4101	snd_data	2505	23	24	35			
4102	rcv_ack	1597	23	24	87	155477168	155510472	155512208
4103	snd_data	2506	23	24	87	155477168	155511920	155512208
4104	rcv_ack	1598	23	24	65535	155478616	155511920	155513656
4105	snd_data	2507	23	24	65535	155478616	155513368	155513656
4106	rcv_ack	1599	23	24	65535	155481512	155513368	155516552
4107	snd_data	2508	23	24	65535	155481512	155514816	155516552
4108	snd_data	2509	23	24	65535	155481512	155516264	155516552
4109	rcv_ack	1600	23	24	64087	155482960	155516264	155518000

当snd_cwnd <= ssthresh时
每收到一个ack，snd_cwnd＋1

当snd_cwnd >ssthresh时
每经过1个RTT，snd_cwnd＋1

慢启动、拥塞避免、拥塞处理和超时重传

- 慢启动算法分析
- 拥塞处理（超时）分析
- 拥塞避免算法分析
- 超时重传机制分析

超时与重传——Karn算法

- 平均往返时延
  - RTT＝α*RTT_old＋（1- α ）RTT_new
  - 重传报文的 RTT_new不计入
  - α＝7/8
- 超时重传时间RTO
  - 正常传输期间：RTO计算算法
    - RTO=β*RTT　　（β>1）
  - 超时重传期间：
    - RTO = γ*RTO_old　　（γ= 2）

Linux中的RTT和RTO

- 按RFC1122规定将RTT和RTO的初始值设置为：RTT=0秒，RTO=3秒。
- 由于Linux这些时间值以10毫秒为单位，实验数据文件中表现为SRTT=0；RTO=300；
- 且RTO的最小值为200毫秒，最大值为120秒。实验数据文件中表现为20 和12000；
- Linux中RTT最小80毫秒，实验数据文件中表现为8

超时和重传

- 超时重传时间：RTO
- 平均往返时延：RTT
- 修正的Karn算法
- 超时重传的算法和时间规律
  - 分析send3***和send4***，主要注意分析超时后重传的时间规律
  - 比较两种链路带宽下的算法的不同